UNIVERSIDAD DR. JOSÉ MATÍAS DELGADO
FACULTAD DE ECONOMÍA, EMPRESA Y NEGOCIOS (FEEN)
LICENCIATURA EN GESTIÓN TECNOLÓGICA Y ANALÍTICA DE DATOS

INFORME TÉCNICO

Plan de Soporte, Mantenimiento y Consolidación de Arquitectura/BD de SumaControl

Esquema de soporte L1/L2/L3 y SLAs, plan de mantenimiento en cuatro dimensiones, Arquitectura e Infraestructura (Sección 4) y Persistencia de Datos (Sección 6) del Documento Maestro

Asignatura	Metodologías de Desarrollo de Software 0 (MDS 0) | Ciclo 2-2026
Unidad Didáctica 3	Implementación y Mejora Continua de los Sistemas de Información — Semana 15
Actividad	Trabajo grupal — 10 puntos (Tema 3: Plan de Soporte Técnico y Mantenimiento del Sistema)
Catedrática	Licda. Ana Lissette Girón Huezo
Grupo	1-1
Integrantes	Andrea Yamilette Aguilera Recinos Fátima Esmeralda Calderón Rivas Otto Alexander Hernández Arévalo
Versión documentada	SumaControl v2.0.1-qa
Repositorio	https://github.com/202400664-ux/proyecto-integrador/tree/main/docs
Fecha	Noviembre de 2026

1. Introducción y alcance

Este informe define cómo se da soporte y mantenimiento a SumaControl una vez en operación y consolida los artefactos técnicos de arquitectura y de datos del Documento Maestro. Tiene cuatro partes:

- **Sección 2:** esquema de soporte técnico con niveles L1, L2 y L3, matriz RACI, matriz de severidad con SLAs, canales de reporte y procedimiento de contingencia.
- **Sección 3:** plan de mantenimiento correctivo, adaptativo, evolutivo/perfectivo y preventivo, y el registro de deuda técnica conocida.
- **Sección 4 (Sección 4 del Documento Maestro):** vista de arquitectura en 4 capas, diagrama de componentes UML, diagrama de despliegue UML y trazabilidad con los casos de uso, diagramas de secuencia y diagrama de clases del proyecto.
- **Sección 5 (Sección 6 del Documento Maestro):** modelo de datos de MongoDB, DER lógico, diccionario de datos de las 12 colecciones y validadores $jsonSchema.

> **Tecnologías realmente implementadas**
>
> **Base de datos:** SumaControl usa MongoDB con Mongoose, no una base relacional. Por eso el modelo se presenta como un DER lógico de colecciones con referencias (PK = _id, FK = referencia por ObjectId) y el equivalente del script DDL (schema.sql) es schema_mongodb.js, con los validadores $jsonSchema de cada colección derivados de los esquemas Mongoose. Las menciones a SQL del enunciado se interpretan como consultas e índices de MongoDB.
>
> **Comunicación:** la interfaz se comunica con la API por HTTP/REST sobre TCP/IP y el tablero se actualiza por consulta periódica (polling cada 5 segundos). La versión implementada no usa WebSockets, por lo que los diagramas documentan ese mecanismo.

2. Esquema de soporte técnico y niveles de atención

El soporte es escalonado: cada nivel resuelve lo que puede dentro de su SLA y, si no, escala al siguiente con el registro del caso (número de pedido, mensaje exacto, hora y captura). L1 es siempre el punto de entrada para los usuarios.

2.1 Niveles de atención

	L1 — Garita / Mesa de ayuda	L2 — Infraestructura / Servidor	L3 — Desarrollo / Algoritmos
Responsable	Equipo de MDS 0, con el líder de turno de garita como apoyo (soporte presencial durante los primeros 5 días del go-live)	Equipo de GSO 0 (servidores) y GRE 1 (redes)	Equipo de PON 0 (API e interfaz) y EDA 0 (motor de cola)
Alcance	Atención a operadores de garita: dudas de uso, reseteo de claves y cuentas, interpretación de A TIEMPO / TARDÍO / ANTICIPADO, mensajes del catálogo de errores y registro manual de contingencia.	Caídas de red o VLAN en la garita, servicio de la API Express detenido, MongoDB inalcanzable o sin conjunto de réplica, certificados y proxy, respaldos y restauración, reinicios controlados.	Mantenimiento de código, errores en RN-04 (tolerancia) y RN-08 (exclusividad del Gateway 5), fallos del motor Heap (EDA 0), correcciones de datos y hotfixes.

	L1 — Garita / Mesa de ayuda	L2 — Infraestructura / Servidor	L3 — Desarrollo / Algoritmos
Canal de recepción	Presencial en garita, chat interno del equipo y registro del caso como Issue en GitHub.	Llamada o mensaje directo (S1/S2), alerta de monitoreo e Issue en GitHub.	Escalamiento desde L2 e Issue en GitHub etiquetado por severidad, con rama bugfix/ o hotfix/.
SLA de respuesta	< 15 minutos	< 30 minutos	< 1 hora
SLA de solución	< 30 minutos (en primer contacto o escalado)	< 2 horas	< 4 horas (hotfix)
Escala a	L2 si es infraestructura; L3 si es comportamiento de reglas	L3 si la causa es de código o datos	Administrador del sistema para aprobar el despliegue

2.2 Matriz RACI del soporte

R = responsable de ejecutar, A = rinde cuentas (uno por actividad), C = consultado, I = informado.

Actividad	Operador / Coordinador	L1 (MDS 0)	L2 (GSO 0 / GRE 1)	L3 (PON 0 / EDA 0)	Administrador
Reportar el incidente y registrar la llegada manual si el sistema cae	R	A	I	—	I
Registrar, clasificar por severidad y dar seguimiento al caso	I	R/A	C	C	I
Reseteo de claves y dudas de uso	I	R/A	—	—	C
Diagnóstico de red, API y base de datos; restablecer el servicio	I	C	R/A	C	I
Corregir bugs en RN-04, RN-08 o el motor Heap	I	C	C	R/A	I
Aprobar y desplegar un hotfix	I	I	R	R	A
Ejecutar el rollback (RTO < 15 min)	I	I	R	C	A
Comunicar al usuario y cerrar el caso	I	R	C	C	A

2.3 Matriz de severidad y SLAs

L1 asigna la severidad al abrir el caso y la reevalúa si cambia el impacto. Los tiempos son los de la matriz de soporte del proyecto para SumaControl; la atención inicial siempre la da L1 en menos de 15 minutos y el nivel asignado responde dentro del tiempo de su severidad.

Severidad	Criterio y ejemplos en SumaControl	Nivel	Canal de reporte	Atención inicial	Solución o workaround	Contingencia
S1 Crítico	El sistema no permite operar: API o MongoDB caídos, no se puede iniciar sesión ni registrar llegadas, ninguna bahía asigna pedidos.	L2 / L3	Llamada o mensaje directo a L2 con aviso a L3; Issue después	< 15 min	< 1 hora	Rollback con RTO < 15 min (2.4) y registro manual en papel en la garita.

Severidad	Criterio y ejemplos en SumaControl	Nivel	Canal de reporte	Atención inicial	Solución o workaround	Contingencia
S2 Alto	Falla grave en una función principal: RN-04 o RN-08 aplican mal, una bahía no finaliza la descarga, la cola no se reevalúa.	L3	Mensaje al canal de soporte e Issue con etiqueta S2	< 30 min	< 4 horas	Asignación y liberación manual de bahías desde el tablero; hotfix.
S3 Medio	Inconsistencia menor o lentitud: consulta de historial o de indicadores lenta, notificación duplicada, mensaje confuso.	L2 / L3	Issue en GitHub etiquetado S3	< 2 horas	< 24 horas	Revisión de índices o de la consulta; corrección en rama bugfix/.
S4 Bajo	Consulta de uso, mejora cosmética o sugerencia del buzón (por ejemplo, el texto de un botón).	L1	Buzón de sugerencias o Issue etiquetado S4	< 4 horas	< 72 horas	Registro en el Product Backlog evolutivo para la siguiente versión.

2.4 Procedimiento de contingencia y rollback (RTO < 15 minutos)

1. **Detectar y decidir (≤ 3 min):** L2 confirma el síntoma, revisa la Auditoría y los eventos recientes, y el Administrador autoriza el rollback si el fallo coincide con un despliegue reciente (activadores del plan de contingencia del plan de implementación).
2. **Contener (en paralelo):** la garita pasa al registro manual en papel con la hora exacta de cada llegada.
3. **Volver a la versión estable (≤ 8 min):** en el servidor, git checkout de la etiqueta anterior, npm ci, npm run build y reinicio de la API y la interfaz. Si el problema son datos, restaurar el último respaldo con mongorestore --drop.
4. **Verificar (≤ 4 min):** iniciar sesión, comprobar que Gateways muestre las 5 bahías, registrar una llegada de prueba y ejecutar npm test en backend/.
5. **Cerrar:** cargar al sistema las llegadas registradas en papel, comunicar a los usuarios y abrir el Issue de causa raíz para L3.

3. Plan de mantenimiento de software

Una estrategia por cada dimensión del mantenimiento, aplicada a la arquitectura real de SumaControl (Next.js, Express, MongoDB y motor EDA 0). Todo cambio pasa por el mismo flujo de control de versiones: rama propia, Pull Request con revisión de un par, pruebas en verde y etiqueta de versión.

3.1 Mantenimiento correctivo

1. **Recepción:** todo reporte entra como Issue con plantilla: pasos para reproducir, mensaje exacto, número de pedido, hora, captura, rol del usuario y severidad.
2. **Reproducción y diagnóstico:** L3 reproduce el caso en el sandbox (QA) y consulta la Auditoría y la bitácora inmutable de eventos para reconstruir lo ocurrido con el pedido.

3. **Aislamiento:** la corrección se hace en una rama bugfix/<issue>-<descripción> creada desde la rama estable, con una prueba que reproduce el fallo antes de arreglarlo.
4. **Verificación:** pruebas unitarias del backend (14 en PASS como base) más la regresión de los casos RN-04 y RN-08 (escenarios E1 a E5 de la capacitación) y revisión de código en el Pull Request.
5. **Hotfix:** para S1 y S2 se crea hotfix/<issue> desde la etiqueta en producción, se despliega tras la aprobación del Administrador, se etiqueta una versión de parche (por ejemplo v2.0.2) y se fusiona de vuelta a main. Si falla la verificación, se aplica el rollback de la sección 2.4.

3.2 Mantenimiento adaptativo

Cambio del entorno	Procedimiento de ajuste
Actualización del navegador en garita (Chrome, Edge o Firefox)	Antes de actualizar el puesto, probar los flujos de inicio de sesión, Caseta de Arribos y Gateways en el sandbox con la nueva versión; actualizar primero un equipo piloto y mantener el anterior como respaldo.
Cambios de sistema operativo o de Node.js en el servidor	Mantener Node.js 22 LTS (mínimo 20.9 por Next.js 16); ante una nueva LTS, instalar en QA, ejecutar npm ci y npm test, y desplegar en la ventana de mantenimiento.
Reconfiguración de IP, VLAN o dominio	La interfaz llama a http://localhost:4000/api: al cambiar direcciones se ajusta esa URL o se publica la API detrás del proxy en el mismo dominio. Actualizar también MONGO_URI y las ACL para que 4000 y 27017 solo se abran entre los nodos que corresponden.
Nuevas versiones de dependencias o parches de seguridad	Revisar con npm audit cada mes, actualizar en una rama aparte y validar en QA antes de fusionar. Regenerar los package-lock.json con registry.npmjs.org.
Cambio de versión de MongoDB	Probar schema_mongodb.js y la suite de integración (MONGO_TEST_URI) contra la nueva versión en QA y respaldar con mongodump antes de actualizar el conjunto de réplica.

3.3 Mantenimiento evolutivo / perfectivo

- **Meta de latencia (API REST < 200 ms):** medir con un middleware de tiempo de respuesta y reportar el percentil 95 de los endpoints críticos (llegadas, gateways, cola y descargas). Si un endpoint supera la meta, L3 abre un Issue de rendimiento.
- **Consultas e índices NoSQL:** Descarga, Evento y Notificacion ya tienen índices compuestos. Pedido solo tiene el índice único de numeroPedido, aunque la cola y la liberación de anticipados filtran por activo, estado, fechaEntradaCola e inicioVentana. schema_mongodb.js propone un índice compuesto para cada consulta; se adoptan solo si explain('executionStats') muestra que reducen los documentos examinados.
- **Motor EDA 0 y tablero:** el heap se reconstruye en cada reevaluación solo con los pedidos en espera y se mantiene con lecturas lean(). Se evaluará reducir el polling de 5 s con respuestas condicionales, o WebSockets como evolución futura, solo si el indicador de latencia lo justifica.
- **Usabilidad en React:** priorizar mejoras con la encuesta de satisfacción y el buzón de sugerencias de la capacitación (por ejemplo, búsqueda de pedido en el tablero y confirmación antes de acciones irreversibles), entregadas por versión y con las guías actualizadas.

3.4 Mantenimiento preventivo

Frecuencia	Rutina	Responsable
Diaria	Revisar los errores de la API y la bitácora de Auditoría y eventos; comprobar que la API, la interfaz y MongoDB estén activos; respaldo automático con mongodump (cron) y retención de los últimos 7 respaldos.	L2
Semanal	Restaurar el último respaldo en el sandbox con mongorestore para probar que sirve; revisar el estado del conjunto de réplica (rs.status()) y el espacio en disco; revisar los indicadores logísticos.	L2 y Coordinador
Mensual	Revisar tamaño de colecciones e índices (db.collection.stats()) y, si procede, compactar en ventana de mantenimiento (compact); npm audit; revisar cuentas de usuario y confirmar que no existan las cuentas de prueba @test.com en producción.	L2 y L3
Continuo	Monitorear la memoria y la CPU del proceso Node.js que ejecuta el motor Heap, con alerta si el consumo crece de forma sostenida; rotar los registros del servidor; cambiar JWT_SECRET al cambiar de ambiente.	L2

3.5 Registro de deuda técnica conocida

Hallazgos de la consolidación que se atienden por medio del plan de mantenimiento, con su severidad y decisión:

Hallazgo	Severidad	Decisión	Tipo de mantenimiento
El tipo de evento se escribe DESCARGADA_INICIADA en el modelo Evento y en descargaController	S4	Se conserva y se documenta tal cual: la bitácora es inmutable y los eventos ya registrados usan ese valor. Si se corrige, se agrega DESCARGA_INICIADA al enum sin eliminar el valor anterior.	Correctivo planificado
La interfaz tiene fija la dirección http://localhost:4000/api	S3	Mover la dirección a una variable de entorno de Next.js antes de separar los nodos de producción.	Adaptativo
El intervalo de aging se define por variable de entorno y no por parámetro	S4	Moverlo a la colección parametros para que lo ajuste el Administrador.	Perfectivo
Los package-lock.json apuntan a un registro espejo de npm	S3	Regenerarlos con registry.npmjs.org y auditar las dependencias.	Preventivo

4. Sección 4 del Documento Maestro: Arquitectura e Infraestructura

4.1 Vista de arquitectura en 4 capas

SumaControl sigue la arquitectura híbrida en capas con módulo reactivo propuesta por la cátedra: tres capas tradicionales (presentación, negocio y persistencia) y una capa algorítmica, el motor EDA 0, que reordena la cola de prioridad cada vez que cambia el estado del patio.

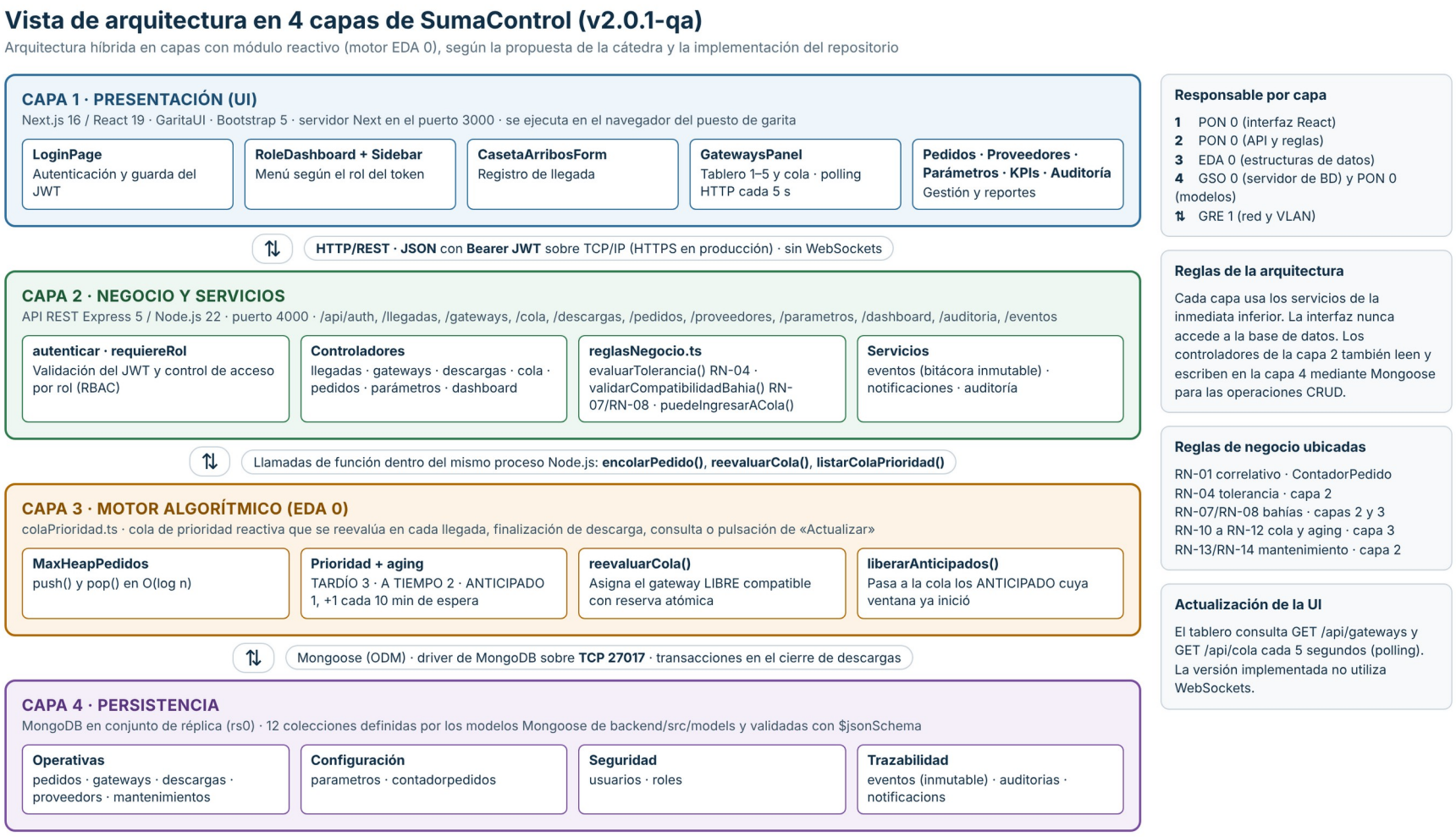


Figura 1. *Vista de arquitectura en 4 capas de SumaControl v2.0.1-qa.*

Capa	Responsabilidad	Depende de	Mecanismo de comunicación	Área
1. Presentación	Captura de datos, visualización del tablero y control de acceso visual por rol	Capa 2 (endpoints REST)	HTTP/REST con JWT; polling 5 s	PON 0
2. Negocio y servicios	Autenticación, RBAC, reglas RN-01 a RN-14, bitácora y notificaciones	Capas 3 y 4	Llamadas de función; Mongoose	PON 0
3. Motor algorítmico	Prioridad con aging, asignación de bahías compatibles y liberación de anticipados	Capa 4 (pedidos y gateways)	Mongoose (findOneAndUpdate atómico)	EDA 0
4. Persistencia	Almacenamiento, validación $jsonSchema, índices y transacciones	Servidor de BD	Driver MongoDB · TCP 27017	GSO 0 / PON 0

4.2 Diagrama de componentes UML

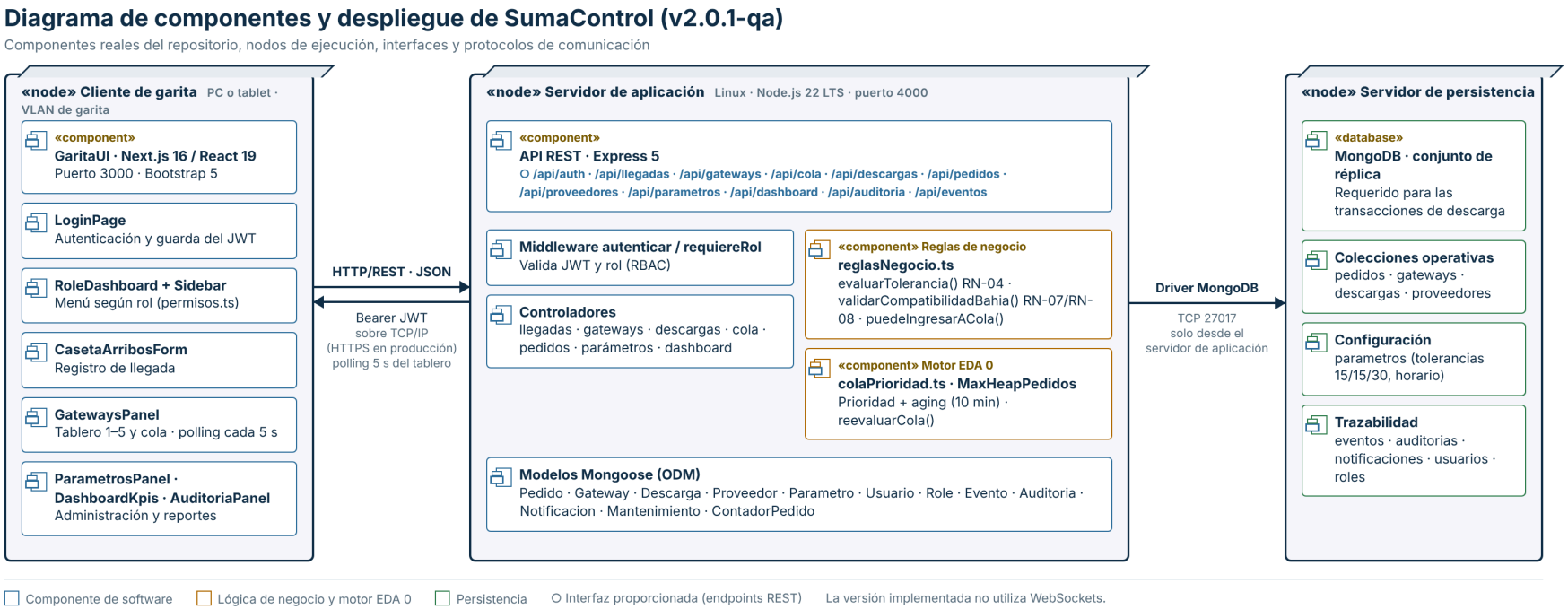


Figura 2. *Diagrama de componentes UML de SumaControl con sus interfaces REST, dependencias y protocolos.*

Componente	Responsabilidad	Interfaz proporcionada	Interfaz requerida
GaritaUI (Next.js)	Interfaz del operador, coordinador y administrador	Interfaz web en el puerto 3000	API REST /api/*
API REST (Express)	Recibe peticiones, valida el JWT y el rol y orquesta los controladores	/api/auth, /llegadas, /gateways, /cola, /descargas, /pedidos, /proveedores, /parametros, /dashboard, /auditoria, /eventos	reglasNegocio, Motor EDA 0, Modelos
reglasNegocio	Funciones puras de RN-01, RN-04, RN-07 y RN-08	evaluarTolerancia(), validarCompatibilidadBahia(), puedeIngresarACola()	—
Motor EDA 0 (colaPrioridad)	Cola de prioridad con aging y asignación de bahías	encolarPedido(), reevaluarCola(), listarColaPrioridad()	reglasNegocio, Modelos
Modelos Mongoose	Esquemas, validaciones e índices de las 12 colecciones	Pedido, Gateway, Descarga, Proveedor y demás modelos	Driver MongoDB

4.3 Diagrama de despliegue UML

El diagrama modela los nodos físicos y virtuales del despliegue de producción definido en el plan de implementación: el puesto de garita, el servidor de aplicación (Express y Next.js) y el servidor de base de datos, con los artefactos que se instalan en cada uno y sus rutas de comunicación.

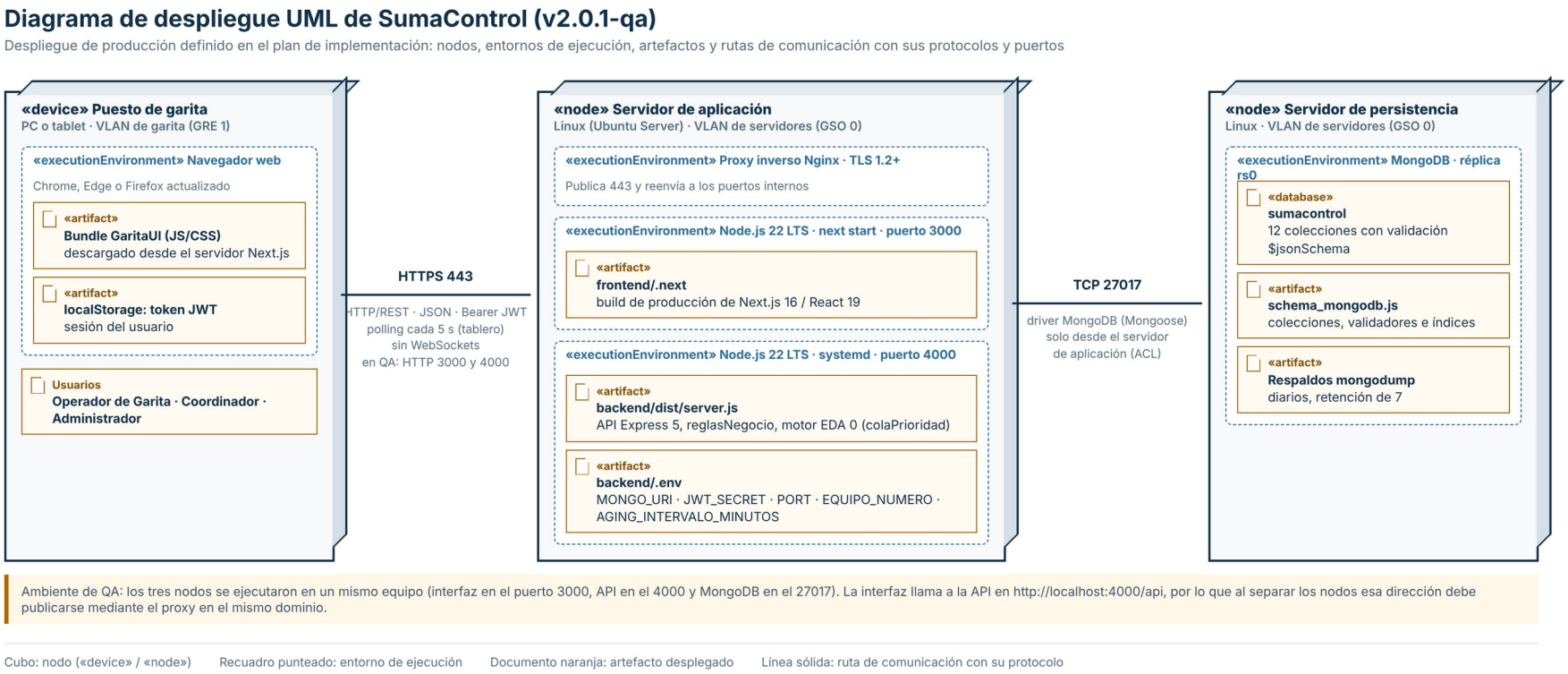


Figura 3. *Diagrama de despliegue UML: puesto de garita, servidor de aplicación y servidor de persistencia, con protocolos y puertos.*

Enlace	Protocolo	Puerto	Observación
Puesto de garita → Servidor de aplicación	HTTPS (HTTP/REST con JSON)	443 (proxy); 3000 y 4000 internos	Token JWT en cada petición; el tablero consulta cada 5 s (polling). Sin WebSockets.
Servidor de aplicación → Servidor de persistencia	TCP/IP, driver MongoDB (Mongoose)	27017	Solo desde el servidor de aplicación (ACL). MongoDB en conjunto de réplica para las transacciones de descarga.
Infraestructura de red (GRE 1)	VLAN de garita, VLAN de servidores y VLAN de gestión	—	La garita solo alcanza el puerto 443; la administración por SSH solo desde la VLAN de gestión.

4.4 Trazabilidad con los artefactos del proyecto

La arquitectura conserva los nombres usados en los casos de uso, los diagramas de secuencia y el diagrama de clases, de modo que cada caso de uso se puede seguir hasta su componente, su endpoint y su colección:

Caso de uso	Componente de interfaz	Endpoint REST	Clase o servicio	Colecciones
CU-01 Programar cita y registrar proveedores	PedidoForm, ProveedorForm	POST /api/pedidos, POST /api/proveedores	pedidoController, generarNumeroPedido()	pedidos, proveedors, contadorpedidos
CU-02 Registrar llegada en garita	CasetaArribosForm	POST /api/llegadas	llegadasController, evaluarTolerancia()	pedidos, eventos, auditorias
CU-03 Asignar gateway y cola	GatewaysPanel	GET /api/cola, POST /api/cola/reevaluar, POST /api/descargas/*	colaPrioridad (MaxHeapPedidos), descargaController	pedidos, gateways, descargas, eventos
CU-04 Reportes de cumplimiento	DashboardKpis, AuditoriaPanel	GET /api/dashboard/kpis, GET /api/auditoria, GET /api/eventos	dashboardController	pedidos, descargas, eventos, auditorias
CU-05 Administrar gateways	GatewaysPanel (Administrador), ParametrosPanel	PUT /api/gateways/:id, /api/parametros	gatewayController, parametroController	gateways, parametros, eventos
CU-06 Notificaciones	GatewaysPanel (avisos operativos)	GET /api/notificaciones	servicio notificaciones	notificacions

El diagrama de secuencia de CU-03 corresponde a la ruta POST /api/llegadas → llegadasController → encolarPedido() → reevaluarCola() → Gateway.findOneAndUpdate(), y las clases del diagrama de clases (Pedido, Gateway, Descarga, Proveedor, Usuario) son los modelos Mongoose de la sección 5.

5. Sección 6 del Documento Maestro: Persistencia de Datos

5.1 Modelo de datos y convenciones

La base es MongoDB y cada entidad es una colección definida por un esquema Mongoose en backend/src/models. Las claves primarias son _id (ObjectId generado por MongoDB) y las relaciones son referencias por ObjectId; no hay claves foráneas impuestas por el motor, así que la integridad la mantienen

los esquemas, los validadores y la lógica de negocio. Las colecciones operativas incluyen activo (baja lógica) y cuatro campos de auditoría de registro. Evento es inmutable: el modelo bloquea actualizaciones y borrados.

Mongoose nombra cada colección con el nombre del modelo en minúsculas y en plural, agregando «s» a los nombres terminados en consonante o vocal; solo Evento declara su colección de forma explícita. Por eso dos nombres no siguen el plural del español:

Modelo	Colección	Modelo	Colección
Pedido	`pedidos`	Usuario	`usuarios`
Gateway	`gateways`	Role	`roles`
Descarga	`descargas`	Evento	`eventos (explícita)`
Proveedor	`proveedors`	Auditoria	`auditorias`
Parametro	`parametros`	Notificacion	`notificacions`
Mantenimiento	`mantenimientos`	ContadorPedido	`contadorpedidos`

5.2 Diagrama entidad-relación (DER lógico)

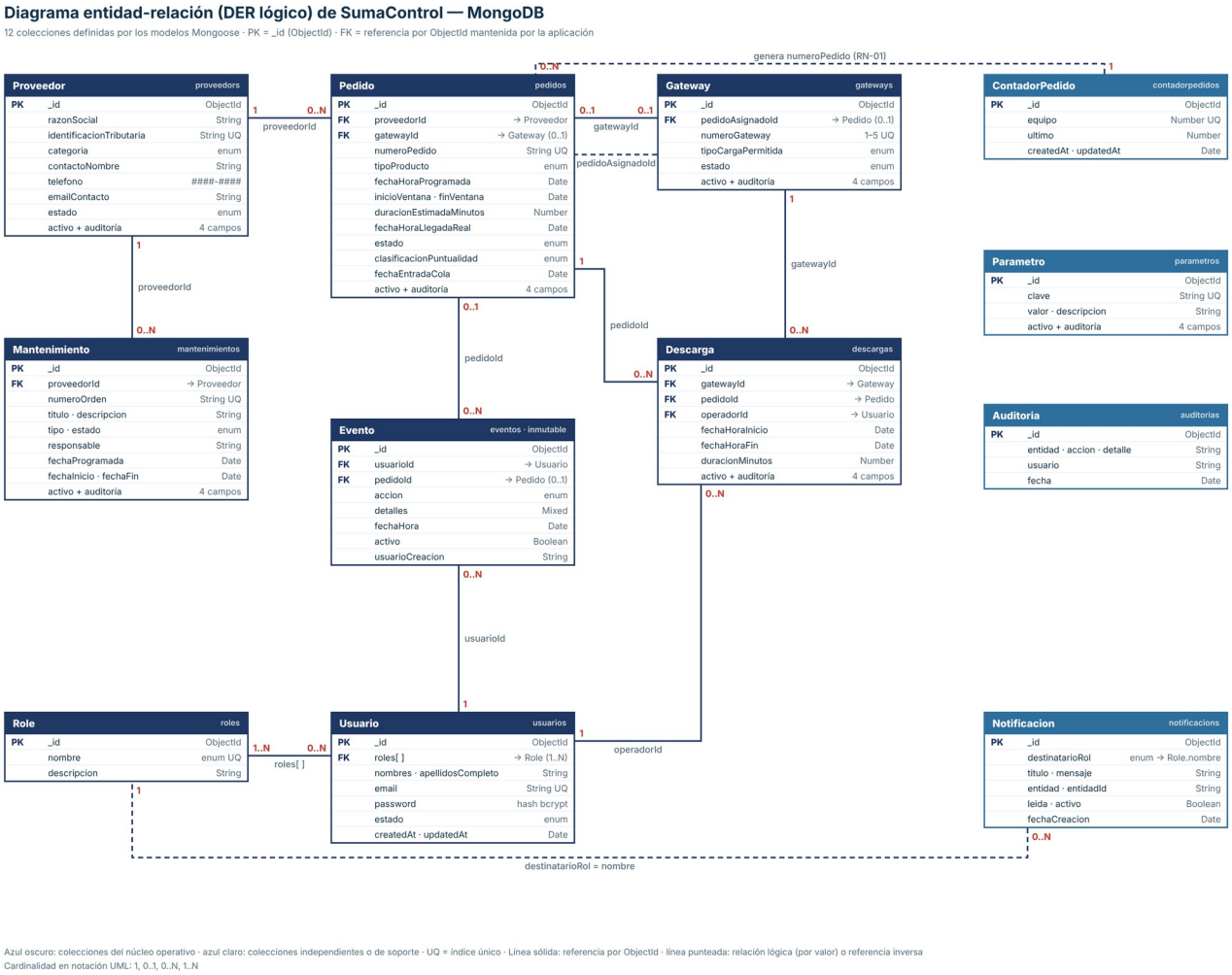


Figura 4. *DER lógico de SumaControl con las 12 colecciones, sus claves y cardinalidades.*

Relaciones: un Proveedor tiene muchos Pedidos y muchas órdenes de Mantenimiento; un Pedido se asigna a lo sumo a un Gateway (Pedido.gatewayId y, en sentido inverso, Gateway.pedidoAsignadoId, que el motor mantiene sincronizados); cada Descarga une un Pedido, un Gateway y el Usuario operador; un Usuario tiene uno o más Role; el Evento registra al Usuario y, si aplica, el Pedido. ContadorPedido genera el correlativo de Pedido.numeroPedido (RN-01) y Notificacion se dirige a un rol por su nombre (relaciones lógicas). Auditoria y Parametro son independientes.

5.3 Diccionario de datos estructurado

Abreviaturas: PK = clave primaria, FK = referencia a otra colección, UQ = índice único, Req = obligatorio. Los cuatro campos de auditoría de registro (usuarioCreacion, usuarioActualizacion, fechaCreacion y fechaActualizacion) se repiten en las colecciones operativas.

Pedido · colección pedidos

Cita de entrega de un proveedor y su ciclo de vida desde la programación hasta la descarga.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
`_id`	ObjectId	PK	Automático	Identificador del pedido.
`numeroPedido`	String	Req, UQ, trim	No nulo	Código correlativo, por ejemplo PED-EQUI04-000000021 (RN-01).
`proveedorId`	ObjectId	FK → Proveedor, Req	No nulo	Proveedor dueño de la cita.
`tipoProducto`	String	Req, enum: construcción | general	No nulo	Tipo de carga; decide el gateway compatible (RN-07, RN-08).
`fechaHoraProgramada`	Date	Req	No nulo	Fecha y hora acordada de la cita.
`inicioVentana`	Date	Req	No nulo	Inicio de la ventana de entrega.
`finVentana`	Date	Req	No nulo	Fin de la ventana de entrega.
`duracionEstimadaMinutos`	Number	Req, mínimo 1	No nulo	Duración estimada de la descarga.
`fechaHoraLlegadaReal`	Date	—	Nulo hasta que llega	Hora real de llegada tomada del servidor (RN-04).
`estado`	String	Enum: PROGRAMADO, ANTICIPADO, A TIEMPO, TARDÍO, AUSENTE, EN_COLA, GATEWAY_ASIGNADO, DESCARGANDO, FINALIZADO, CANCELADO	Defecto PROGRAMADO	Etapa actual del pedido.
`clasificacionPuntualidad`	String	Enum: ANTICIPADO | A TIEMPO | TARDÍO	Nulo hasta registrar la llegada	Puntualidad original; prioridad base en la cola.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
gatewayId	ObjectId	FK → Gateway	Nulo	Bahía asignada al pedido.
fechaEntradaCola	Date	—	Nulo	Ingreso a la cola; base del aging y del desempate.
activo	Boolean	Req	Defecto true	Baja lógica: false oculta el registro sin borrarlo.
usuarioCreacion	String	Req	Defecto «Sistema»	Usuario que creó el registro.
usuarioActualizacion	String	Req	Defecto «Sistema»	Último usuario que modificó el registro.
fechaCreacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de creación del documento.
fechaActualizacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de la última modificación.

Gateway · colección gateways

Bahía de descarga: del 1 al 4 para carga general y el 5 para construcción.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
_id	ObjectId	PK	Automático	Identificador de la bahía.
numeroGateway	Number	Req, UQ, 1 a 5	No nulo	Número visible de la bahía.
tipoCargaPermitida	String	Req, enum: general | construcción	No nulo	Carga que puede recibir (RN-08).
estado	String	Req, enum: LIBRE | OCUPADO | FUERA DE SERVICIO	Defecto LIBRE	Estado mostrado con el código cromático.
pedidoAsignadoId	ObjectId	FK → Pedido	Nulo	Pedido reservado para la bahía.
activo	Boolean	Req	Defecto true	Baja lógica: false oculta el registro sin borrarlo.
usuarioCreacion	String	Req	Defecto «Sistema»	Usuario que creó el registro.
usuarioActualizacion	String	Req	Defecto «Sistema»	Último usuario que modificó el registro.
fechaCreacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de creación del documento.
fechaActualizacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de la última modificación.

Descarga · colección descargas

Registro de una descarga en una bahía, con su duración.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
`_id`	ObjectId	PK	Automático	Identificador de la descarga.
`gatewayId`	ObjectId	FK → Gateway, Req; índice con fechaHoraFin	No nulo	Bahía usada.
`pedidoId`	ObjectId	FK → Pedido, Req; índice con fechaHoraFin	No nulo	Pedido descargado.
`operadorId`	ObjectId	FK → Usuario, Req	No nulo	Operador que la inició.
`fechaHoraInicio`	Date	Req	No nulo	Inicio de la descarga.
`fechaHoraFin`	Date	—	Nulo mientras está en curso	Fin de la descarga.
`duracionMinutos`	Number	Mínimo 0	Nulo hasta finalizar	Duración registrada al finalizar.
`activo`	Boolean	Req	Defecto true	Baja lógica: false oculta el registro sin borrarlo.
`usuarioCreacion`	String	Req	No nulo, sin defecto	Usuario que creó el registro.
`usuarioActualizacion`	String	Req	No nulo, sin defecto	Último usuario que lo modificó.
`fechaCreacion`	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de creación del documento.
`fechaActualizacion`	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de la última modificación.

Proveedor · colección proveedors

Empresa que entrega mercancía en el centro de distribución.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
`_id`	ObjectId	PK	Automático	Identificador del proveedor.
`razonSocial`	String	Req, trim, solo letras y espacios	No nulo	Nombre legal.
`identificacionTributaria`	String	Req, UQ, trim, solo dígitos	No nulo	Identificación tributaria.
`categoria`	String	Req, enum: construcción | general	No nulo	Categoría de carga que entrega.
`contactoNombre`	String	Req, trim, solo letras y espacios	No nulo	Persona de contacto.
`telefono`	String	Req, trim, formato ####-####	No nulo	Teléfono de contacto.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
emailContacto	String	Req, minúsculas, formato de correo	No nulo	Correo de contacto.
estado	String	Req, enum: activo | inactivo	Defecto activo	Estado funcional del proveedor.
activo	Boolean	Req	Defecto true	Baja lógica: false oculta el registro sin borrarlo.
usuarioCreacion	String	Req	Defecto «Sistema»	Usuario que creó el registro.
usuarioActualizacion	String	Req	Defecto «Sistema»	Último usuario que modificó el registro.
fechaCreacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de creación del documento.
fechaActualizacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de la última modificación.

Mantenimiento · colección mantenimientos

Orden de mantenimiento programada y su seguimiento.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
_id	ObjectId	PK	Automático	Identificador de la orden.
numeroOrden	String	Req, UQ, trim	No nulo	Número de la orden de mantenimiento.
proveedorId	ObjectId	FK → Proveedor, Req	No nulo	Proveedor relacionado con la orden.
titulo	String	Req, trim	No nulo	Título breve de la orden.
descripcion	String	Req, trim	No nulo	Detalle del trabajo.
tipo	String	Req, enum: preventivo | correctivo | predictivo	No nulo	Tipo de mantenimiento.
estado	String	Req, enum: pendiente | en_progreso | completado | cancelado	Defecto pendiente	Avance de la orden.
responsable	String	Req, trim	No nulo	Persona a cargo.
fechaProgramada	Date	Req	No nulo	Fecha prevista.
fechaInicio · fechaFin	Date	—	Nulo	Inicio y cierre reales.
observaciones	String	trim	Nulo	Notas del cierre.
activo	Boolean	Req	Defecto true	Baja lógica: false oculta el registro sin borrarlo.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
usuarioCreacion	String	Req	Defecto «Sistema»	Usuario que creó el registro.
usuarioActualizacion	String	Req	Defecto «Sistema»	Último usuario que modificó el registro.
fechaCreacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de creación del documento.
fechaActualizacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de la última modificación.

Parametro · colección parametros

Configuración editable por el Administrador: tolerancias RN-04 y RN-05 y horario operativo.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
_id	ObjectId	PK	Automático	Identificador del parámetro.
clave	String	Req, UQ, trim	No nulo	Por ejemplo tolerancia_anticipado_minutos.
valor	String	Req	No nulo	Valor (minutos o HH:MM).
descripcion	String	Req	No nulo	Explicación del parámetro.
activo	Boolean	Req	Defecto true	Baja lógica: false oculta el registro sin borrarlo.
usuarioCreacion	String	Req	Defecto «Sistema»	Usuario que creó el registro.
usuarioActualizacion	String	Req	Defecto «Sistema»	Último usuario que modificó el registro.
fechaCreacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de creación del documento.
fechaActualizacion	Date	Automático (timestamps)	Lo genera Mongoose	Fecha de la última modificación.

ContadorPedido · colección contadorpedidos

Contador atómico del correlativo de pedidos, un documento por equipo (RN-01).

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
_id	ObjectId	PK	Automático	Identificador del contador.
equipo	Number	Req, UQ	No nulo	Número de equipo (por ejemplo 4).

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
`ultimo`	Number	Req	Defecto 0	Último correlativo emitido.
`createdAt · updatedAt`	Date	Automático	Lo genera Mongoose	Fechas técnicas del documento.

Usuario · colección usuarios

Persona con acceso al sistema.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
`_id`	ObjectId	PK	Automático	Identificador del usuario.
`nombres`	String	Req, trim	No nulo	Nombres.
`apellidosCompleto`	String	Req, trim	No nulo	Apellidos.
`email`	String	Req, UQ, minúsculas, trim	No nulo	Correo de inicio de sesión.
`password`	String	Req	No nulo	Contraseña guardada como hash bcrypt.
`roles`	ObjectId[ ]	FK → Role, Req	No nulo	Uno o más roles; definen el menú y las acciones.
`estado`	String	Req, enum: activo | inactivo	Defecto activo	Habilita o no el acceso.
`createdAt · updatedAt`	Date	Automático	Lo genera Mongoose	Fechas de creación y modificación.

Role · colección roles

Rol de seguridad para el control de acceso (RBAC).

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
`_id`	ObjectId	PK	Automático	Identificador del rol.
`nombre`	String	Req, UQ, minúsculas, enum: administrador | coordinador | operador	No nulo	Nombre del rol.
`descripcion`	String	Req, trim	No nulo	Alcance del rol.

Evento · colección eventos

Bitácora inmutable de eventos operativos (solo inserción).

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
`_id`	ObjectId	PK	Automático	Identificador del evento.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
usuarioId	ObjectId	FK → Usuario, Req, inmutable	No nulo	Usuario autenticado que provocó el evento.
pedidoId	ObjectId	FK → Pedido, inmutable; índice con fechaHora	Nulo	Pedido relacionado, si aplica.
accion	String	Req, inmutable, enum: LLEGADA_REGISTRADA, GATEWAY_ASIGNADO, DESCARGADA_INICIADA, DESCARGA_FINALIZADA, CITA_REPROGRAMADA, GATEWAY_MANTENIMIENTO	No nulo	Tipo de evento (DESCARGADA_INICIADA se escribe así en el código; ver 3.5).
detalles	Mixed	Req, inmutable	No nulo	Datos del evento en formato JSON.
fechaHora	Date	Req, inmutable; índice descendente	Defecto fecha actual	Cuándo ocurrió.
activo	Boolean	Req, inmutable	Defecto true	Indicador de vigencia.
usuarioCreacion	String	Req, inmutable	No nulo	Nombre del usuario que registró el evento.
fechaCreacion	Date	Automático	Lo genera Mongoose	Fecha de creación (no hay fecha de actualización).

Auditoria · colección auditorias

Historial operativo de cambios (entidad, acción y usuario).

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
_id	ObjectId	PK	Automático	Identificador del registro.
entidad	String	Req, trim	No nulo	Entidad afectada (Gateway, Cola, Llegada…).
accion	String	Req, trim	No nulo	Acción realizada.
detalle	String	Req, trim	No nulo	Descripción del cambio.
usuario	String	Req, trim	Defecto «Sistema»	Quién lo ejecutó.
fecha	Date	Req	Defecto fecha actual	Cuándo ocurrió.
createdAt · updatedAt	Date	Automático	Lo genera Mongoose	Fechas técnicas del documento.

Notificacion · colección notificacions

Aviso interno dirigido a un rol, por ejemplo la asignación de un gateway.

Campo	Tipo	Restricciones	Nulo / defecto	Descripción funcional
`_id`	ObjectId	PK	Automático	Identificador de la notificación.
`destinatarioRol`	String	Req, enum: administrador | coordinador | operador; índice compuesto	No nulo	Rol que la recibe (relación lógica con Role.nombre).
`titulo`	String	Req, trim	No nulo	Título.
`mensaje`	String	Req, trim	No nulo	Texto del aviso.
`entidad`	String	Req	No nulo	Tipo de entidad relacionada.
`entidadId`	String	—	Nulo	Identificador de la entidad relacionada.
`leida`	Boolean	Req	Defecto false	Si ya fue leída.
`activo`	Boolean	Req	Defecto true	Baja lógica: false oculta el registro sin borrarlo.
`fechaCreacion`	Date	Automático	Lo genera Mongoose	Fecha de creación (no hay fecha de actualización).

5.4 Validadores $jsonSchema (equivalente del script DDL)

El archivo schema_mongodb.js, en la carpeta docs/ del repositorio, cumple la función del schema.sql: crea las 12 colecciones con su validador $jsonSchema (tipos BSON, campos obligatorios, enumeraciones y patrones), los índices únicos y los índices compuestos de los modelos. Usa validationLevel «moderate» para no bloquear documentos que ya existen y validationAction «error» para rechazar inserciones inválidas. El script completo está en la carpeta docs/ del repositorio.

Elemento del script	Contenido
Ejecución	mongosh "mongodb://localhost:27017/sumacontrol?replicaSet=rs0" schema_mongodb.js
Índices únicos (8, además del _id)	proveedors.identificacionTributaria, pedidos.numeroPedido, gateways.numeroGateway, parametros.clave, roles.nombre, usuarios.email, mantenimientos.numeroOrden y contadorpedidos.equipo
Índices de consulta de los modelos (5)	descargas (gatewayId, fechaHoraFin) y (pedidoId, fechaHoraFin); eventos (fechaHora) y (pedidoId, fechaHora); notificacions (destinatarioRol, activo, fechaCreacion)
Índices propuestos (2)	pedidos (activo, estado, fechaEntradaCola) y (activo, estado, inicioVentana), marcados en el script como opcionales hasta medir su efecto (sección 3.3)
Verificación	El script se ejecutó con una base simulada: crea las 12 colecciones y 15 índices sin errores, y sus patrones son expresiones regulares válidas. Antes de producción se ejecuta en QA y se confirma con show collections.

6. Enlaces y contenido del repositorio

Carpeta de documentación	https://github.com/202400664-ux/proyecto-integrador/tree/main/docs
Contenido	Plan_Soporte_Mantenimiento_Arquitectura_BD_SumaControl.pdf (este informe) diagrama_despliegue_uml.png diagrama_componentes_uml.png arquitectura_4_capas.png der_sumacontrol.png schema_mongodb.js (validadores $jsonSchema e índices; equivalente del schema.sql)
Versión documentada	https://github.com/202400664-ux/proyecto-integrador/tree/v2.0.1-qa
Fuente de los modelos	backend/src/models (12 modelos Mongoose)

Fuentes

- Repositorio oficial de SumaControl, versión v2.0.1-qa: modelos Mongoose, controladores, motor de cola y configuración; base de las secciones 3.5, 4 y 5.
- Matriz de soporte L1-L2-L3 de SumaControl (equipo MDS 0): tiempos de respuesta y solución por nivel y por severidad; base de las secciones 2.1 y 2.3.
- Material de cátedra de la Unidad 3 (arquitectura de sistemas, diagramas de componentes y despliegue, DER y diccionario de datos); base de las secciones 4 y 5.

- Informes del equipo «Plan de Implementación, Despliegue y Mantenimiento» y «Plan de Capacitación y Guías de Usuario»: activadores de contingencia, plan de despliegue y catálogo de errores; base de las secciones 2.4, 3 y 4.3.